Unit 4.3: Feature Justification and Model Comparison

DAIR-3 Workshop, Summer 2026

Presented by Suraj Rampure (rampure@umich.edu)

Overview

- In Unit 4.2, we learned how to build many candidate feature pipelines.
- Today, we'll discuss two philosophies for choosing among them:
    - **Approach 1: Predictive performance**. Which candidate model predicts best on data it did not train on?
    - **Approach 2: Feature diagnostics**. Which candidate features are redundant, unstable, or hard to justify?
- We'll start with commute times, then connect the same ideas to the iBudget report and the birthweight data.

Approach 1: Cross-validation

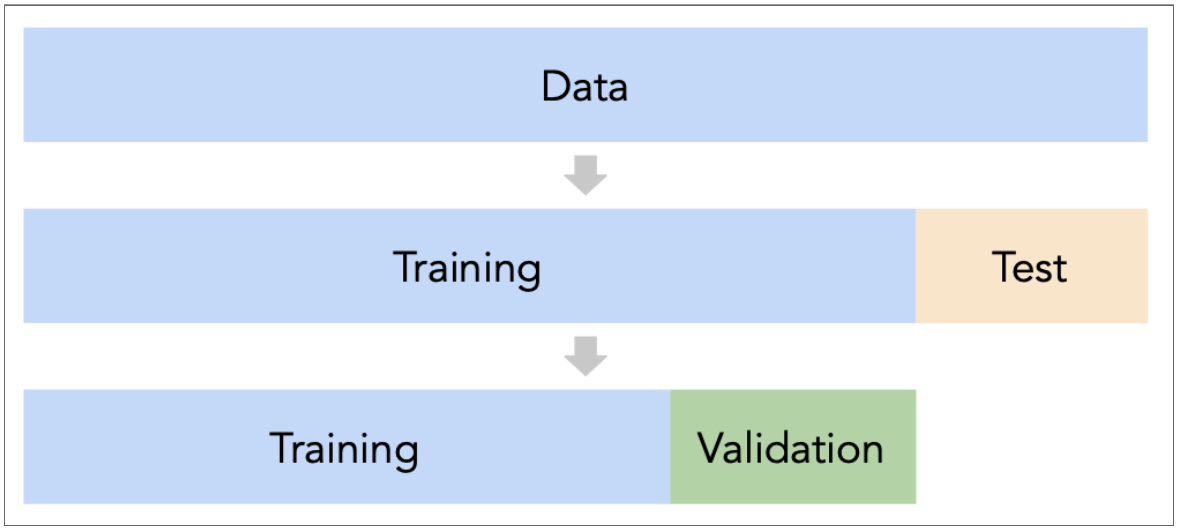
The first recipe: a single train/test split

- Suppose we're choosing between many different models.
- We won't know whether a model has **overfit** to our sample unless we get to see how well it performs on a new sample from the same population.
- Idea: **split** our dataset into a **training set** and **test set**.

- For each model we're considering:
  - Use **only** the training set to fit that model, i.e. find w^*'s.
  - Use the test set to evaluate that model's error, e.g. compute its MSE.
- Pick the model with the **best** test set performance.
- **What is wrong with this approach?**

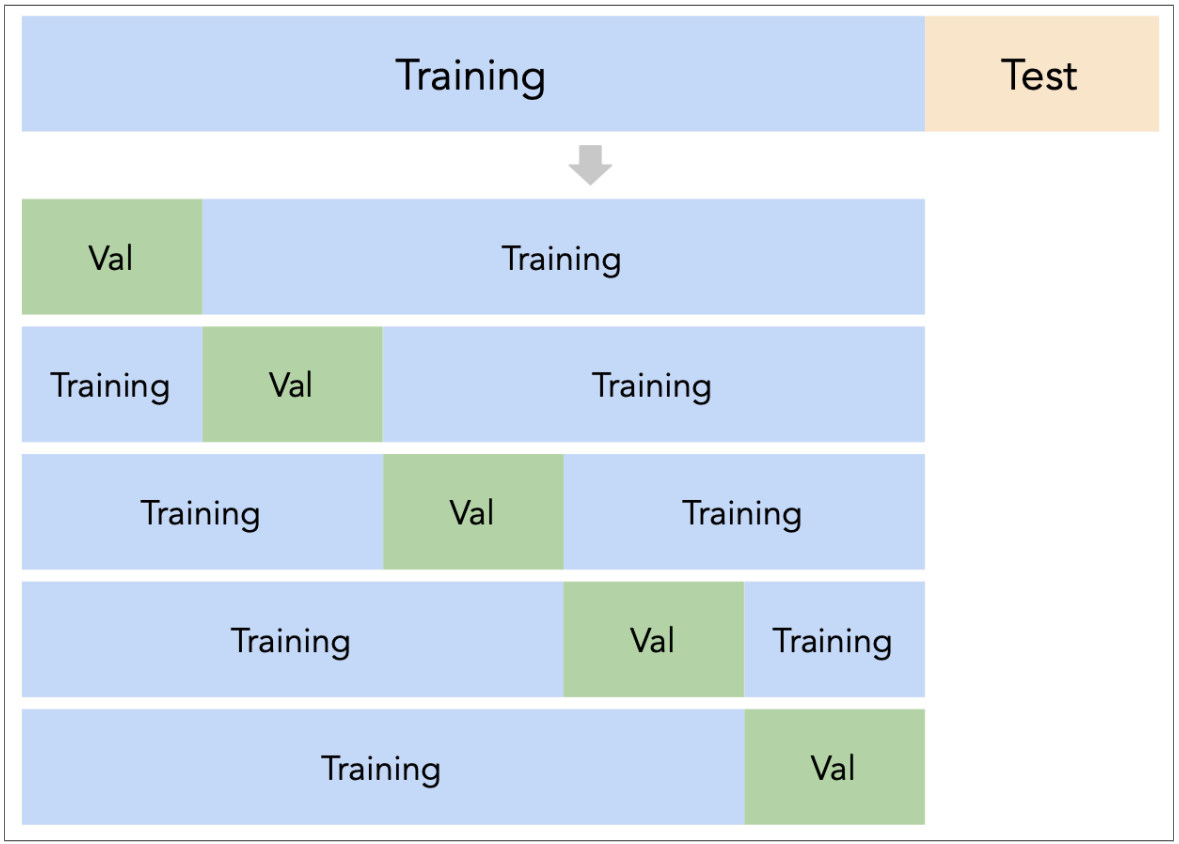
Another idea: a single validation set

- One way to avoid overfitting to the test data is to further split the training data into training and validation.



- **Issue**: this approach depends too much on the specific slice of data that happened to be in the validation set, which is a small piece of the overall dataset.

The ultimate solution: k-fold cross-validation



- In k-fold cross-validation, each training row gets used for validation once and for training $k-1$ times.
- k is the number of "slices" (called "folds") that the training set is partitioned into.
- We choose the candidate model with the best **average** validation performance.

Warm-up: Polynomial degree

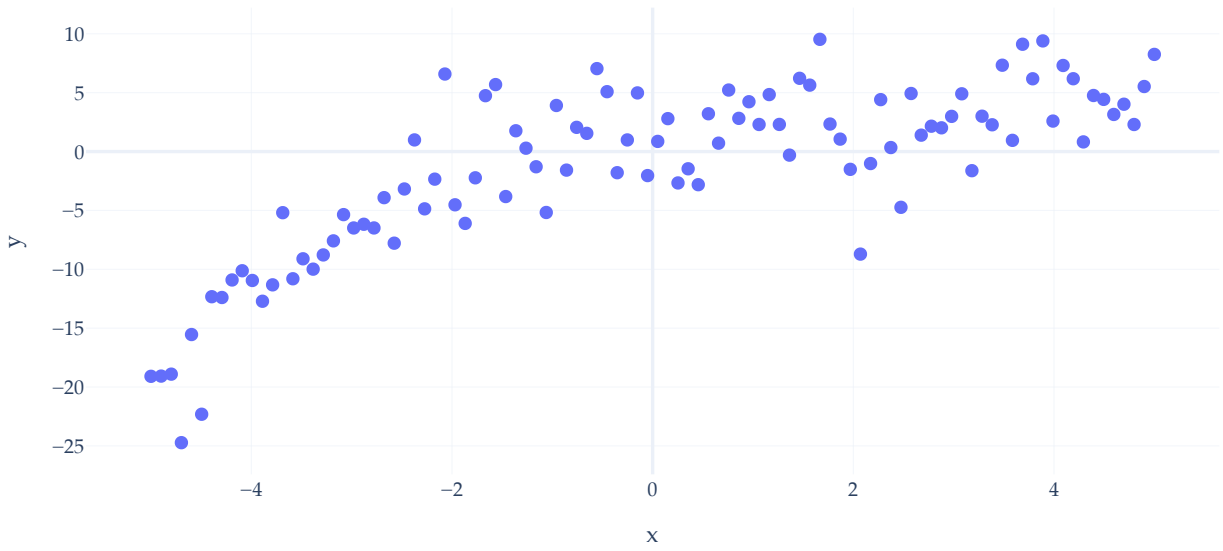
- Before comparing full pipelines, let's return to the synthetic polynomial regression example from Unit 4.1.
- The candidate models differ by one hyperparameter: the polynomial degree.

In [2]:

```
sample_1 = make_polynomial_sample(n=100, random_state=23)

fig = px.scatter(
    sample_1,
    x="x",
    y="y",
    title="Synthetic Sample 1 from Notebook 1",
)
fig
```

Synthetic Sample 1 from Notebook 1



Train/test split first

- Before choosing a polynomial degree, set aside a test set.
- Cross-validation will happen only inside the training set.
- The test set comes back **only after the degree has been chosen**.

In [3]:

```
X_poly = sample_1[["x"]]
y_poly = sample_1["y"]

X_poly_train, X_poly_test, y_poly_train, y_poly_test = train_test_split(
    X_poly,
    y_poly,
    test_size=0.2,
    random_state=23,
)

# Until we choose a degree, we should not look at X_poly_test and y_poly_test.
```

An implementation of 5-fold cross validation

- The code below performs 5-fold cross validation using the `GridSearchCV` object.

  Notice that we didn't need to write any loops or shuffling logic ourselves.

  How do you pick k? This is not something that is "optimized": 5 or 10 are often chosen in practice, though there are bias/variance tradeoff and computational implications.

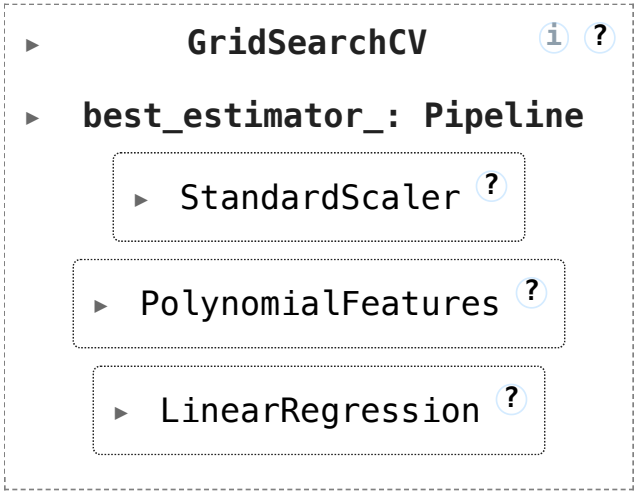
In [4]:

```
degrees = np.arange(1, 31)
poly_cv = KFold(n_splits=5, shuffle=True, random_state=23)

poly_search = GridSearchCV(
    make_pipeline(StandardScaler(), PolynomialFeatures(include_bias=False), LinearRegression()),
    param_grid={"polynomialfeatures__degree": degrees},
    scoring="neg_mean_squared_error",
    cv=poly_cv,
)

poly_search.fit(X_poly_train, y_poly_train)
```

Out[4]:



- What does this mean? Let's visualize the results. The code below is purely for exploration.

In [5]:

```
split_cols = [f"split{i}_test_score" for i in range(5)]
poly_cv_scores = (
    pd.DataFrame(poly_search.cv_results_)
    .assign(degree=lambda df: df["param_polynomialfeatures__degree"].astype(int))
    .set_index("degree")[split_cols]
    .mul(-1)
    .T
)
poly_cv_scores.index = [f"Fold {i}" for i in range(1, 6)]
```

A 5-by-30 cross-validation grid

- Suppose we want to choose a degree from 1 through 30, using 5-fold cross-validation on the training set.
- **30 columns:** the 30 candidate polynomial degrees.
- **5 rows:** the 5 validation folds.
- **Each cell:** fit that degree on 4 folds, then compute validation mean squared error (MSE) on the held-out fold.
- `GridSearchCV` fits $30 \times 5 = 150$ models before we pick a degree.

In [6]:

```
poly_cv_scores.round(2)
```

Out[6]:

degree	1	2	3	4	5	6	7	8	9	10	...	21
Fold 1	21.32	10.92	9.84	9.81	9.62	10.48	10.57	12.31	11.59	11.86	...	66.19
Fold 2	31.66	17.17	12.10	12.44	12.31	11.93	12.24	12.23	13.11	11.96	...	13.33
Fold 3	10.62	8.12	9.92	10.72	12.08	12.23	12.34	11.99	11.97	10.92	...	15.96
Fold 4	10.39	11.81	8.15	8.16	8.39	8.76	9.43	9.65	8.54	6.99	...	168.18
Fold 5	32.95	24.84	21.55	21.55	21.39	21.43	21.98	22.20	22.02	20.09	...	19.62

5 rows × 30 columns

- We should choose the degree with the **lowest average validation MSE**.

Average across folds

- For each degree, average the 5 validation errors in that column.
- The degree with the smallest average validation error is the cross-validation choice.

In [7]:

```
poly_cv_scores.mean(axis=0)
```

Out[7]:

```
degree
1          21.389395
2          14.572474
3          12.312280
4          12.536678
5          12.760120
6          12.964476
7          13.313738
8          13.680114
9          13.446653
10         12.363930
11         12.905874
12         12.623359
13         14.069612
14         16.530185
15         14.723595
16         14.831484
17         14.708848
18         20.812007
19         41.800106
20         49.154782
21         56.656316
22        101.146390
23        151.818091
24       1171.253640
25       1488.053851
26       2157.422357
27       2442.402849
28      50474.621984
29      81856.812142
30     461157.620945
dtype: float64
```

In [8]:

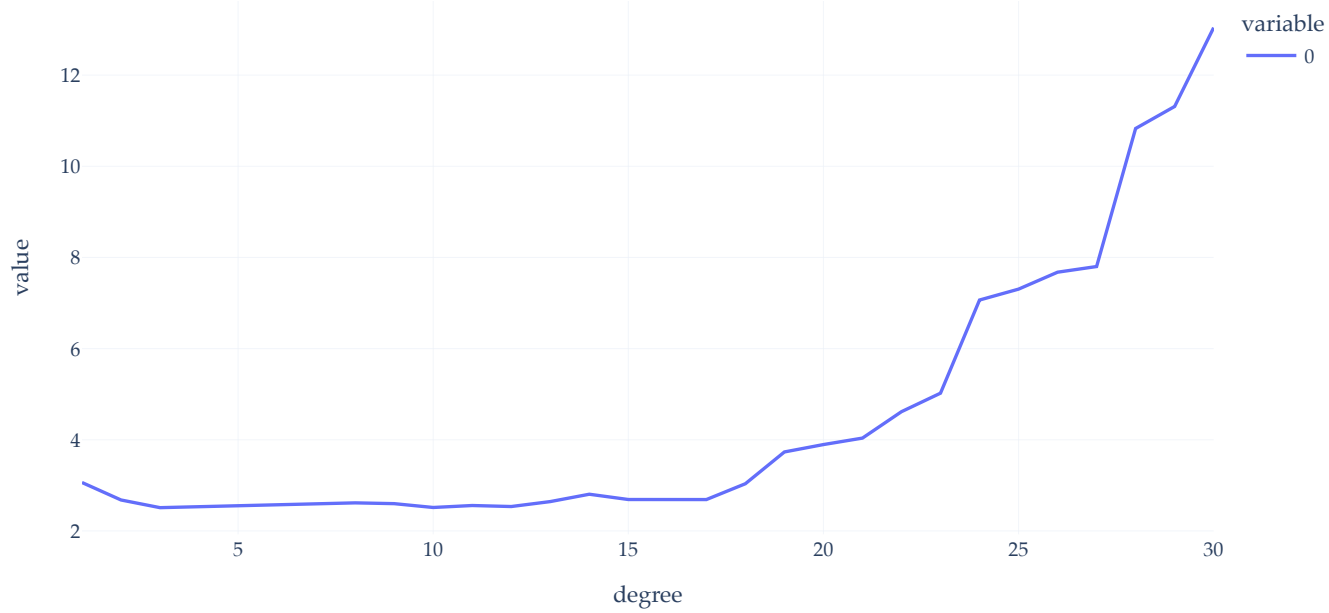
```
poly_cv_scores.mean(axis=0).sort_values()
```

Out[8]:

```
degree
3          12.312280
10         12.363930
4          12.536678
12         12.623359
5          12.760120
11         12.905874
6          12.964476
7          13.313738
9          13.446653
8          13.680114
13         14.069612
2          14.572474
17         14.708848
15         14.723595
16         14.831484
14         16.530185
18         20.812007
1          21.389395
19         41.800106
20         49.154782
21         56.656316
22        101.146390
23        151.818091
24       1171.253640
25       1488.053851
26       2157.422357
27       2442.402849
28      50474.621984
29      81856.812142
30     461157.620945
dtype: float64
```

In [9]:

```
np.log(poly_cv_scores.mean(axis=0)).plot(kind='line')
```


variable
0
value
degree
2
4
6
8
10
12
5
10
15
20
25
30

Key takeaways

- Using cross-validation, we deduce that a degree 3 polynomial is most likely to perform well on unseen data.
- Is a degree 3 polynomial, trained on the entire training set, **guaranteed** to be the best performing model on the test set? **No!** We expect it to, though.
- Interesting observation: if we **didn't** perform $k = 5$-fold cross validation, and instead just picked one of the five folds as a validation set, each fold would have led to a different optimal degree!

In [10]:

```
# This is showing us the degree with the lowest validation MSE per fold.
# Degree 3 has the lowest average validation MSE, but not the lowest validation MSE in any one fold.
poly_cv_scores.idxmin(axis=1)
```

Out[10]:

```
Fold 1     5
Fold 2    20
Fold 3     2
Fold 4    13
Fold 5    28
dtype: int64
```

Example: Commute time `Pipelines`

- Let's return to the commute times dataset.
- Instead of choosing between polynomial degrees, we will choose between entire `Pipeline`s.
- Cross-validation does the same thing either way:
    - fit each candidate repeatedly,
    - score it on validation folds,
    - average the validation scores,
    - choose candidate with best average validation performance.

Setup

In [11]:

```
commutes = pd.read_csv("data/commute-times.csv")
commutes[["date", "day", "month", "day_of_month", "departure_hour", "minutes"]].head()
```

Out[11]:

	date	day	month	day_of_month	departure_hour	minutes
0	5/15/2023	Mon	May	15	10.816667	68.0
1	5/16/2023	Tue	May	16	7.750000	94.0
2	5/22/2023	Mon	May	22	8.450000	63.0
3	5/23/2023	Tue	May	23	7.133333	100.0
4	5/30/2023	Tue	May	30	9.150000	69.0

In [12]:

```
X_commutes_train, X_commutes_test, y_commutes_train, y_commutes_test = train_test_split(
    commutes.drop(columns="minutes"),
    commutes["minutes"],
    random_state=23,
)
```

Instantiating 5 `Pipeline`s

- `departure_hour` only: predict commute time using just the time of day the trip started.
- `departure_hour` + `day_of_month` : add the numeric day of the month, so the model can use simple calendar timing.
- `departure_hour` + `day` OHE (one hot encoded): keep departure hour, and one-hot encode the day of week.
- `departure_hour` + `day` OHE + `month` OHE: add both day-of-week and month indicators, so the model can capture weekly and seasonal patterns.
- poly `hour` + `day` / `month` / `week` OHE: use a degree-2 polynomial in departure hour, plus one-hot encoded day, month, and week-of-month features.

In [13]:

```
week_of_month_encoder = make_pipeline(
    FunctionTransformer(lambda x: ((x - 1) // 7 + 1)),
    OneHotEncoder(handle_unknown="ignore"),
)

pipes = {
    "departure_hour only": make_pipeline(
        make_column_transformer(("passthrough", ["departure_hour"]), remainder="drop"),
        LinearRegression(),
    ),
    "departure_hour + day_of_month": make_pipeline(
        make_column_transformer(("passthrough", ["departure_hour", "day_of_month"]), remainder="drop"),
        LinearRegression(),
    ),
    "departure_hour + day OHE": make_pipeline(
        make_column_transformer(
```

```
            ("passthrough", ["departure_hour"]),
            (OneHotEncoder(handle_unknown="ignore"), ["day"]),
            remainder="drop",
        ),
        LinearRegression(),
    ),
    "departure_hour + day OHE + month OHE": make_pipeline(
        make_column_transformer(
            ("passthrough", ["departure_hour"]),
            (OneHotEncoder(handle_unknown="ignore"), ["day", "month"]),
            remainder="drop",
        ),
        LinearRegression(),
    ),
    "poly hour + day/month/week OHE": make_pipeline(
        make_column_transformer(
            (PolynomialFeatures(2, include_bias=False), ["departure_hour"]),
            (OneHotEncoder(handle_unknown="ignore"), ["day", "month"]),
            (week_of_month_encoder, ["day_of_month"]),
            remainder="drop",
        ),
        LinearRegression(),
    ),
}
```

In [14]:

```
cv = KFold(n_splits=5, shuffle=True, random_state=23)
cv_summary = pd.DataFrame()
for name, pipe in pipes.items():
    scores = cross_validate(
        pipe,
        X_commutes_train,
        y_commutes_train,
        cv=cv,
        scoring={'MSE': 'neg_mean_squared_error'},
        return_train_score=True,
    )

    # cross_validate in sklearn is built to find models that "maximize" a metric.
    # We want to minimize MSE, so we maximize the negative of MSE.
    cv_summary.loc[name, "Average Training MSE"] = -scores["train_MSE"].mean()
    cv_summary.loc[name, "Average Validation MSE"] = -scores["test_MSE"].mean()

cv_summary.sort_values("Average Validation MSE")
```

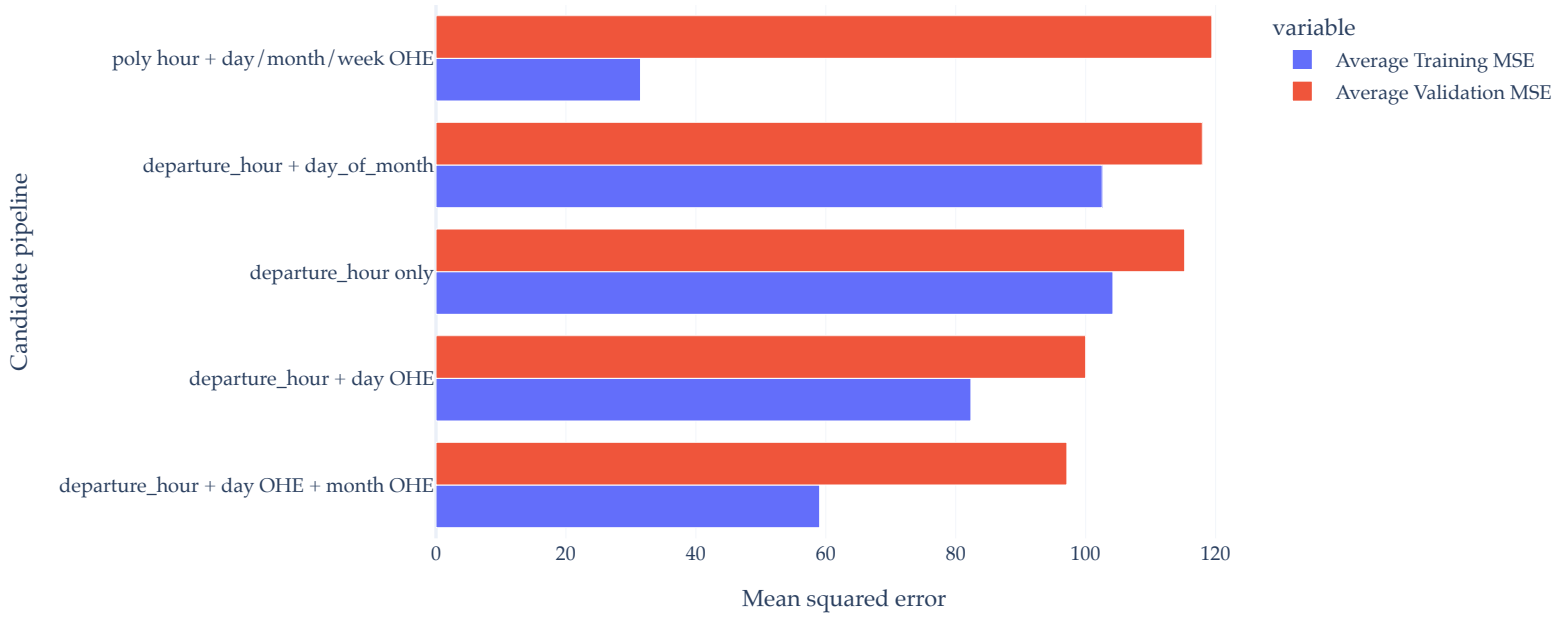
Out[14]:

	Average Training MSE	Average Validation MSE
departure_hour + day OHE + month OHE	59.051866	97.112471
departure_hour + day OHE	82.339720	100.002528
departure_hour only	104.200104	115.241301
departure_hour + day_of_month	102.548659	117.959644
poly hour + day/month/week OHE	31.482806	119.423811

Which model should we choose?

In [15]:

```
fig = (
    cv_summary[["Average Training MSE", "Average Validation MSE"]]
    .sort_values("Average Validation MSE")
    .plot(kind="barh", barmode="group", width=950)
)
fig.update_layout(xaxis_title="Mean squared error", yaxis_title="Candidate pipeline")
fig
```



Summary: Cross-validation

Suppose we have several candidate models that we'd like to choose from. To do so:

1. Split the data into two sets: **training** and **test**.
2. Use only the **training** data when designing, training, and tuning the model.
    - Use k**-fold cross-validation** to choose hyperparameters and estimate the model's ability to generalize.
    - Do not ❌ look at the **test** data in this step!
3. Commit to your final model and train it using the entire **training** set.
4. Test the data using the **test** data. If the performance (e.g. MSE) is not acceptable, return to step 2.
5. Finally, train on **all available data** and ship the model to production! 🚢

Is mean squared error the only measurement of regression model performance?

- No!
- **MSE:** mean squared error; strongly punishes large misses.
- **RMSE**: root mean squared error; same ranking as MSE, but in more interpretable units.
- **MAE:** mean absolute error; easier to explain and less dominated by outliers.
- R^2: **larger** values correspond to better model performance.

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - h(\vec{x}_i))^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2} = 1 - \frac{\text{RSS}}{\text{TSS}}$$

  - If the model is a linear model with an intercept term, **then on the training data**, this is equal to:

$$\begin{aligned} R^2 &= \frac{\text{variance}(\text{predicted } y \text{ values})}{\text{variance}(\text{actual } y \text{ values})} \\ &= [\text{correlation}(\text{actual } y \text{ values}, \text{predicted } y \text{ values})]^2 \end{aligned}$$

  - On training set, ranges from 0 to 1. On test set, could be less than 0 (never more than 1).
  - For linear models, R^2 is a function of MSE.

Cross-validation with various metrics

- Is the model with the lowest average validation MSE the same as the model with the **highest** average validation R^2?

In [16]:

```
scoring = {
    "MSE": "neg_mean_squared_error",
    "MAE": "neg_mean_absolute_error",
    "R2": "r2",
}
cv = KFold(n_splits=5, shuffle=True, random_state=23)

cv_summary = pd.DataFrame()
for name, pipe in pipes.items():
    scores = cross_validate(
        pipe,
        X_commutes_train,
        y_commutes_train,
        cv=cv,
        scoring=scoring,
        return_train_score=True,
    )
    # Feel free to uncomment these.
#     cv_summary.loc[name, "Average Training MSE"] = -scores["train_MSE"].mean()
    cv_summary.loc[name, "Average Validation MSE"] = -scores["test_MSE"].mean()
#     cv_summary.loc[name, "Average Training MAE"] = -scores["train_MAE"].mean()
    cv_summary.loc[name, "Average Validation MAE"] = -scores["test_MAE"].mean()
#     cv_summary.loc[name, "Average Training R^2"] = -scores["train_R2"].mean()
    cv_summary.loc[name, "Average Validation R^2"] = scores["test_R2"].mean()

cv_summary.sort_values("Average Validation MSE").round(2)
```

Out[16]:

	Average Validation MSE	Average Validation MAE	Average Validation R^2
departure_hour + day OHE + month OHE	97.11	7.19	0.28
departure_hour + day OHE	100.00	7.19	0.33
departure_hour only	115.24	7.99	0.27
departure_hour + day_of_month	117.96	8.20	0.25
poly hour + day/month/week OHE	119.42	7.97	0.01

- **Discuss**: Change `n_splits=5` to `n_splits=10`. What happens, and why?

Takeaways from Approach 1

- Cross-validation is useful when the question is:
    - Which candidate workflow predicts best on data it did not train on?
- Cross-validation does **not**, by itself, explain whether the chosen features are redundant, stable, or scientifically meaningful.
- Use it in conjunction with Approach 2.

Approach 2: Feature diagnostics

Feature diagnostics

- Now, let's analyze the relationships among the features themselves to inform our feature selection process.
- This is not a replacement for performance evaluation: both approaches work together.
- It answers a different question:
    - Are some features carrying overlapping information?
    - Are some features hard to justify?
- Remember: simplicity is key. If we don't absolutely need a feature, we may want to remove it.

Variance inflation factor

- Goal: Measure how "redundant" feature $x^{(j)}$ is.
- Suppose our multiple linear regression model has d features, and is of the form

$$h(\vec{x}_i) = w_0 + w_1 x_i^{(1)} + x_i^{(2)} + \ldots + x_i^{(d)} = w_0 + \sum_{j=1}^{d} x_i^{(j)}$$

- The **variance inflation factor (VIF)** of feature j measures **how well feature j can be predicted using the other $d-1$ features** (using a linear model).
    - To compute it, build a linear regression model in which $x^{(j)}$ is the target and all other features are used to predict it.
    - Find the R^2 value of the resulting model; call this R_j^2. Then,

$$\text{VIF}_j = \frac{1}{1 - R_j^2}$$

Interpreting VIF

$$\text{VIF}_j = \frac{1}{1 - R_j^2}$$

- A large VIF (often >5 or >10 are used as rules-of-thumb) means that feature is highly predictable from the other features.
- That is evidence of **multicollinearity**.
- It does **not** automatically mean the feature must be removed.
- It does mean coefficient-level interpretation is less stable, as we discussed in the context of one hot encoding in Unit 4.2, though this doesn't necessarily impact model MSE (for instance).

In [17]:

```
compute_vif(commutes[['departure_hour', 'day_of_month']])
```

Out[17]:

	feature	VIF
0	departure_hour	1.00538
1	day_of_month	1.00538

In [18]:

```
commutes[['departure_hour', 'day_of_month']].corr()
```

Out[18]:

	departure_hour	day_of_month
departure_hour	1.000000	0.073153
day_of_month	0.073153	1.000000

In [19]:

```
1 / (1 - 0.073153 ** 2)
```

Out[19]:

```
1.005380152549729
```

VIF with polynomial features

- Notice that if we create a `departure_hour^2` feature, the VIF rises massively!

In [20]:

```
compute_vif((
    commutes[['departure_hour', 'day_of_month']]
    .assign(**{'departure_hour^2': commutes['departure_hour'] ** 2})
))
```

Out[20]:

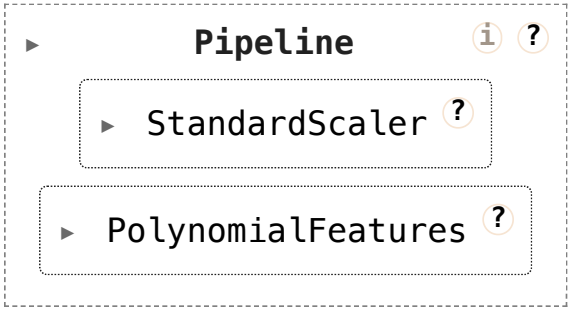
	feature	VIF
0	departure_hour	134.437123
2	departure_hour^2	134.262817
1	day_of_month	1.016763

- Especially when constricted to a domain, the "default" polynomial features of x, x^2, x^3, etc. are often highly correlated.
- One solution: Standardize before constructing polynomial features.

In [21]:

```
p = make_pipeline(StandardScaler(), PolynomialFeatures(degree=2))
p
```

Out[21]:



In [22]:

```
commutes_std_poly = pd.DataFrame(p.fit_transform(commutes[['departure_hour']])[:, 1:], columns=['(std
commutes_std_poly['day_of_month'] = commutes['day_of_month']
commutes_std_poly
```

Out[22]:

	(std dept)	(std dept)^2	day_of_month
0	2.297810	5.279932	15
1	-0.692594	0.479686	16
2	-0.010001	0.000100	22
3	-1.293925	1.674241	23
4	0.672591	0.452378	30
...	...	...	...
60	-0.920124	0.846629	27
61	-0.887620	0.787869	29
62	-0.855115	0.731222	4
63	-0.985133	0.970487	5
64	-0.838863	0.703692	7

65 rows × 3 columns

In [23]:

```
# Much lower, despite using the same information!
compute_vif(commutes_std_poly)
```

Out[23]:

	feature	VIF
1	(std dept)^2	1.081433
0	(std dept)	1.079685
2	day_of_month	1.016763

Pearson and Spearman correlations

- For numeric features, two common checks are:
    - **Pearson correlation:** linear association between two numeric variables. Up until now, anytime we've referred to the "correlation" between two variables, **this is what we've referred to**.
    - **Spearman correlation:** correlation between ranks; useful for **monotone** relationships that may not be linear.
- These are metrics to evaluate on the original, raw data.

In [24]:

```
commutes.select_dtypes('number').corr(method="pearson")
```

Out[24]:

	home_departure_mileage	work_arrival_mileage	work_departure_mileage	home_arrival_mil
home_departure_mileage	1.000000	1.000000	1.000000	1.000
work_arrival_mileage	1.000000	1.000000	1.000000	1.000
work_departure_mileage	1.000000	1.000000	1.000000	1.000
home_arrival_mileage	1.000000	1.000000	1.000000	1.000
minutes	0.003487	0.003598	0.090165	0.090
departure_hour	-0.215962	-0.216014	-0.321173	-0.321
mileage_to_work	0.107876	0.108068	0.068514	0.068
minutes_to_home	0.345094	0.345099	0.345098	0.345
work_departure_time_hr	-0.237351	-0.237358	-0.237375	-0.237
mileage_to_home	0.236790	0.236790	0.236812	0.236
day_of_month	-0.113047	-0.113030	-0.167778	-0.167

In [25]:

```
commutes.select_dtypes('number').corr(method="spearman")
```

Out[25]:

	home_departure_mileage	work_arrival_mileage	work_departure_mileage	home_arrival_mil
home_departure_mileage	1.000000	1.000000	1.000000	1.000
work_arrival_mileage	1.000000	1.000000	1.000000	1.000
work_departure_mileage	1.000000	1.000000	1.000000	1.000
home_arrival_mileage	1.000000	1.000000	1.000000	1.000
minutes	-0.001706	-0.001706	0.085970	0.085
departure_hour	-0.187779	-0.187779	-0.352729	-0.352
mileage_to_work	0.083818	0.083818	-0.120615	-0.120
minutes_to_home	0.359328	0.359328	0.359328	0.359
work_departure_time_hr	-0.133780	-0.133780	-0.133780	-0.133
mileage_to_home	0.140720	0.140720	0.140720	0.140
day_of_month	-0.115117	-0.115117	-0.180319	-0.180

For illustration...

In [26]:

```
commutes_std_poly
```

Out[26]:

	(std dept)	(std dept)^2	day_of_month
0	2.297810	5.279932	15
1	-0.692594	0.479686	16
2	-0.010001	0.000100	22
3	-1.293925	1.674241	23
4	0.672591	0.452378	30
...	...	...	...
60	-0.920124	0.846629	27
61	-0.887620	0.787869	29
62	-0.855115	0.731222	4
63	-0.985133	0.970487	5
64	-0.838863	0.703692	7

65 rows × 3 columns

In [27]:

```
commutes_std_poly.corr(method="pearson")
```

Out[27]:

	(std dept)	(std dept)^2	day_of_month
(std dept)	1.000000	0.25462	0.073153
(std dept)^2	0.254620	1.00000	-0.083420
day_of_month	0.073153	-0.08342	1.000000

In [28]:

```
commutes_std_poly.corr(method="spearman")
```

Out[28]:

	(std dept)	(std dept)^2	day_of_month
(std dept)	1.000000	-0.009661	0.075563
(std dept)^2	-0.009661	1.000000	-0.025042
day_of_month	0.075563	-0.025042	1.000000

In [29]:

```
commutes_std_poly.abs()
```

Out[29]:

	(std dept)	(std dept)^2	day_of_month
0	2.297810	5.279932	15
1	0.692594	0.479686	16
2	0.010001	0.000100	22
3	1.293925	1.674241	23
4	0.672591	0.452378	30
...	...	...	...
60	0.920124	0.846629	27
61	0.887620	0.787869	29
62	0.855115	0.731222	4
63	0.985133	0.970487	5
64	0.838863	0.703692	7

65 rows × 3 columns

In [30]:

```
commutes_std_poly.abs().corr(method="spearman")
```

Out[30]:

	(std dept)	(std dept)^2	day_of_month
(std dept)	1.000000	1.000000	-0.025042
(std dept)^2	1.000000	1.000000	-0.025042
day_of_month	-0.025042	-0.025042	1.000000

What about categorical features?

- Do **not** diagnose categorical relationships by correlating one hot encoded columns!
- For categorical-categorical pairs, a common choice is **Cramer's** V, which ranges from 0 to 1. We won't discuss this further here.

How to use feature diagnostics

- Feature diagnostics can justify keeping, transforming, combining, or removing features.
- Examples:
    - High VIF for `departure_hour` and `departure_hour_squared` says that unless you standardize first, coefficients should not be interpreted separately with much confidence.
    - High VIF for `day_of_month` and `week_number` says they carry overlapping calendar information.
- The point is not to mechanically delete everything with a high number; rather, use these metrics to inform how you may select features.

Aside: Regularization

- We won't discuss this here, but know that another solution to multicollinearity, other than having to run VIF checks to figure out which features to consider drop, is to employ **regularization**.
- In the context of linear regression, the most common form of regularization is "Ridge Regression". This is used in the iBudget study.
- We won't discuss this further: **here** is a lecture on it.

Discuss: Florida's iBudget program

- Open: **https://apd.myflorida.com/resources/reports/APD%20Algorithm%20Report.pdf**
- What metrics do they use to compare the performance of the several new models they developed?
- Did they use cross-validation? If so, how and for what purpose?
- How did they use correlations and VIF to decide which features to keep?

Key takeaways

- Cross-validation chooses among candidate workflows based on held-out performance.
- MSE, MAE, and R^2 answer related but different performance questions.
  MSE and RMSE measure the same thing, but are just in different units.
- VIF asks whether one feature is predictable from the other features.
- Pearson and Spearman are for numeric feature relationships.

Back to the birthweight dataset 👶

Setup

In [31]:

```
births = load_birthweight_1971()
births = births[[
    "birthweight", "sex", "momrace", "momage", "dadage", "plurality", "birthorder"
]].dropna().copy()

births.sample(5, random_state=23)
```

Out[31]:

	birthweight	sex	momrace	momage	dadage	plurality	birthorder
1037139	3345.0	male	White	20	22.0	1	1.0
181863	1503.0	male	White	23	23.0	1	2.0
1701201	3232.0	female	White	26	26.0	1	4.0
91949	3941.0	female	White	38	36.0	1	2.0
738186	2920.0	male	White	44	47.0	1	7.0

Activity: Choose and justify features

- Working with the birthweight data:
    - Propose 2-3 candidate feature sets.
    - Compare them using cross-validation and a metric you can explain.
    - Check for redundant numeric features using correlation and VIF.
    - For categorical variables, inspect the original categories instead of correlating one hot encoded columns.
    - Write one sentence justifying a feature you would keep.
    - Write one sentence identifying a feature that might be redundant or hard to justify.
- Submit proof of your work to ALICE. **Use the prompt here, rather than what is written in ALICE, as your assessment for Unit 4.3.** ALICE mentions something about subgroup model performance checking, which we will look at in Unit 4.4.

In [ ]: